

SAE 1.1 - Implémentation d'un jeu vidéo

Jeu du Taquin

Table des matières :

1. Introduction

1.1. Présentation du projet

1.2. Objectifs pédagogiques

2. Description des fonctionnalités

2.1. Écran de configuration (Menu)

2.2. Écran de jeu

2.3. Écran de victoire

2.4. Contrôles et Interactions

3. Structure du programme

3.1. Fichiers principaux (main.c, Makefile)

3.2. Gestion de l'interface utilisateur (graphique.c, graphique.h)

3.3. Gestion de la logique (jeu.c, jeu.h)

3.4. Répertoire des ressources

4. Données représentant l'état de la partie

4.1. Structure principale Jeu

4.2. Structure de configuration Config

5. Analyse des algorithmes

5.1. Initialisation et Mélange (Garantie de solvabilité)

5.2. Gestion du déplacement et de la souris

6. Conclusion personnelle

1. Introduction

1.1. Présentation du projet

Dans le cadre de la Situation d'Apprentissage et d'Évaluation (SAE) 1.1, nous avons conçu et développé une version numérique du célèbre jeu de réflexion "Le Taquin". Ce jeu de solitaire se présente sous la forme d'un damier composé de tuiles carrées, qui sont en réalité des morceaux d'une image découpée.

Une case de la grille est laissée vide, ce qui permet au joueur de faire coulisser les tuiles adjacentes (horizontalement ou verticalement) vers cet espace libre. L'objectif est de partir d'une configuration mélangée aléatoirement et de parvenir à reconstituer l'image originale en remettant toutes les tuiles dans le bon ordre. Notre version du jeu inclut une interface graphique complète permettant au joueur de choisir son image, de configurer la difficulté (taille de la grille) et de jouer à la souris ou au clavier.

1.2. Objectifs pédagogiques

La réalisation de ce projet avait pour but de consolider nos compétences en programmation structurée et de nous confronter aux exigences d'un développement logiciel complet. Le programme devait respecter plusieurs contraintes techniques strictes :

- **Langage imposé** : Le développement a été effectué exclusivement en langage C, en respectant la norme ANSI (C89).
- **Ressources limitées** : Aucun emprunt de code extérieur ou de bibliothèque tierce n'était autorisé, à l'exception de la bibliothèque graphique fournie par l'IUT pour la gestion de l'affichage et des événements.
- **Architecture logicielle** : Le code devait être modulaire, réparti en plusieurs fichiers sources (.c) et en-têtes (.h), et compilable via un fichier Makefile.
- **Algorithmique** : Il fallait concevoir des structures de données adaptées et implémenter des algorithmes robustes, notamment pour garantir que le mélange du plateau aboutisse toujours à une configuration soluble.

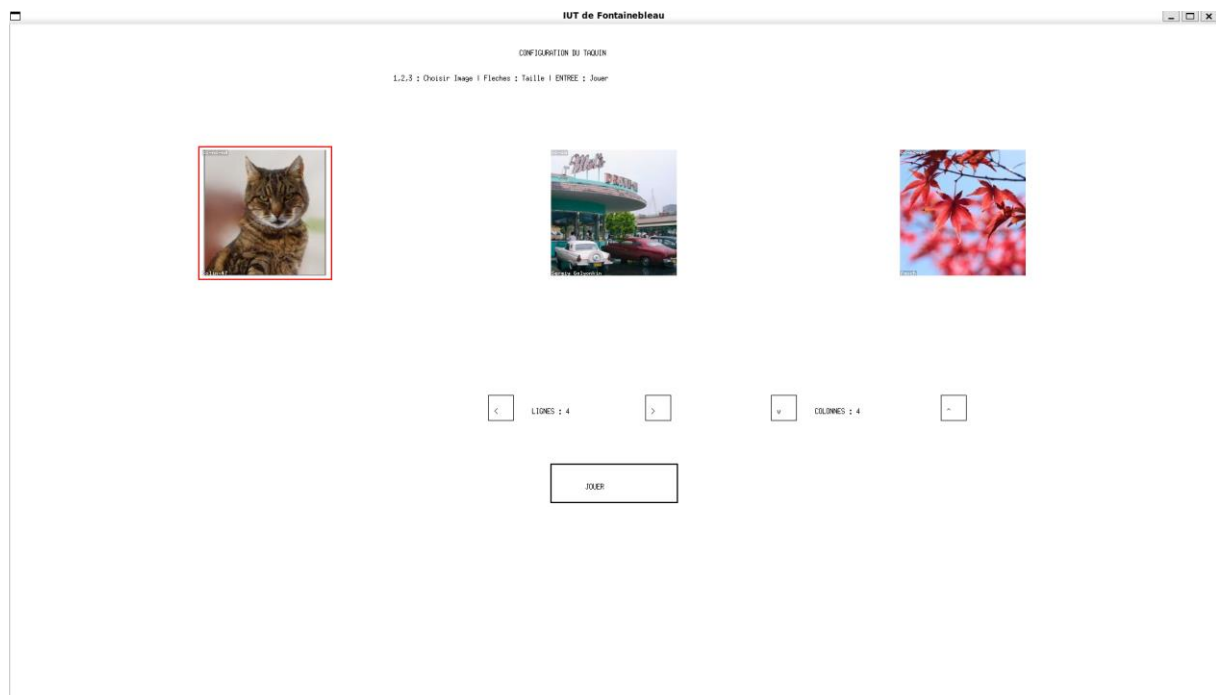
2. Description des fonctionnalités

L'application se décompose en trois écrans principaux qui guident l'utilisateur depuis la configuration jusqu'à la résolution du puzzle. L'interface a été conçue pour être à la fois intuitive et esthétique, en utilisant uniquement les primitives graphiques de la bibliothèque de l'IUT.

2.1. Écran de configuration (Menu)

Au lancement du programme, l'utilisateur est accueilli par le menu de configuration. Cet écran permet de paramétrer la partie avant de la lancer.

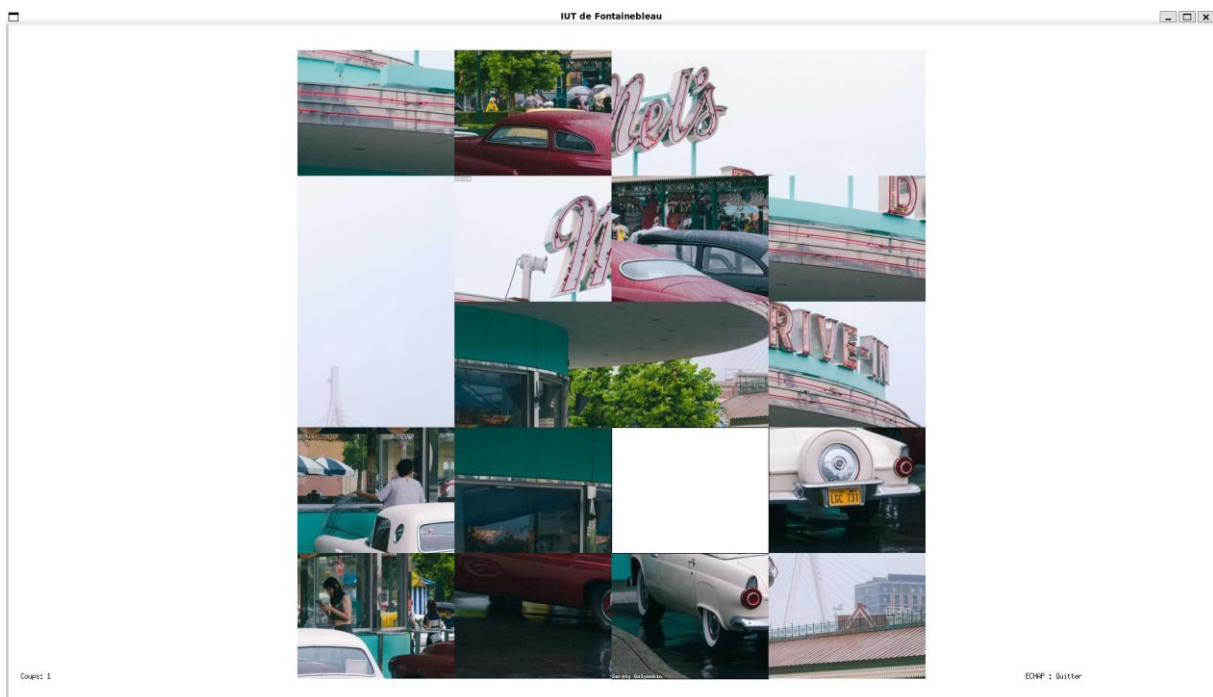
- **Sélection de l'image** : Trois images miniatures sont affichées au centre de l'écran (par exemple : un chat, un personnage de manga, un paysage naturel). L'utilisateur peut choisir l'image qu'il souhaite reconstituer. L'image sélectionnée est mise en évidence par un **cadre rouge** épais.
- **Sélection de la difficulté** : En dessous des images, deux sélecteurs permettent de définir les dimensions de la grille (nombre de **Lignes** et de **Colonnes**). Les valeurs sont comprises entre 3 et 8. Des flèches directionnelles visuelles (<, >, ^, v) permettent de modifier ces valeurs.
- **Validation** : Un bouton "JOUER", situé en bas de l'écran, permet de valider les choix et de lancer la partie.



2.2. Écran de jeu

Une fois la partie lancée, la fenêtre se redimensionne automatiquement pour s'adapter à la taille de la grille choisie, garantissant ainsi un affichage optimal quel que soit le nombre de tuiles.

- **Le Plateau :** L'image choisie est découpée en tuiles carrées mélangées. Une case noire représente l'espace vide nécessaire aux déplacements.
- **Le Modèle :** Pour aider le joueur, une miniature de l'image originale (le modèle à reconstituer) reste affichée en permanence dans le coin supérieur gauche de la fenêtre, accompagnée de la mention "Modèle".
- **Le Score :** Un compteur "Coups : X" est affiché en bas de l'écran et s'actualise en temps réel à chaque déplacement valide effectué par le joueur.



2.3. Écran de victoire

Le programme vérifie l'état du puzzle après chaque mouvement. Lorsque le joueur parvient à remettre toutes les tuiles dans l'ordre (de gauche à droite et de haut en bas, avec la case vide en dernière position), la partie s'arrête.

Un message de félicitations "**BRAVO ! GAGNÉ !**" s'affiche alors distinctement au centre de l'écran, sur un fond rouge, pour marquer la réussite du joueur. Le jeu attend ensuite une action de l'utilisateur pour quitter.

2.4. Contrôles et Interactions

Afin d'offrir une expérience utilisateur fluide, toutes les fonctionnalités du jeu sont accessibles via deux modes d'interaction simultanés :

- **La Souris :**
 - Dans le menu : Clic sur les miniatures pour choisir l'image, clic sur les flèches pour changer la taille, clic sur le bouton "JOUER".
 - En jeu : Un simple clic sur une tuile adjacente à la case vide permet de la déplacer vers cet espace.
- **Le Clavier :**
 - Dans le menu : Les touches 1, 2, 3 sélectionnent l'image. Les flèches directionnelles modifient le nombre de lignes et colonnes. La touche Entrée lance le jeu.
 - En jeu : Les flèches directionnelles permettent de déplacer la case vide (Haut, Bas, Gauche, Droite).
 - Général : La touche Echap permet de fermer l'application à tout moment.

3. Structure du programme

Le programme est organisé de manière modulaire pour une meilleure séparation des responsabilités et une maintenance plus aisée. Chaque fichier source (.c) et d'en-tête (.h) correspond à une fonctionnalité ou un ensemble de fonctionnalités spécifiques.

Voici une présentation détaillée de la structure du projet :

3.1. Fichiers principaux

- main.c
 - Point d'entrée du programme.
 - Il joue le rôle de chef d'orchestre : il initialise les composants (graphique et logique), lance la boucle du menu de configuration, puis la boucle principale du jeu.
 - Il gère l'enchaînement logique entre les différentes phases (Menu -> Jeu -> Victoire -> Fin).
- Makefile
 - Permet la compilation automatisée du programme.
 - Contient les cibles but (pour compiler l'exécutable), run (pour compiler et lancer le jeu directement), et clean (pour supprimer les fichiers objets .o et l'exécutable généré).
 - Gère les dépendances et les liens avec la bibliothèque graphique et X11.

3.2. Gestion de l'interface utilisateur

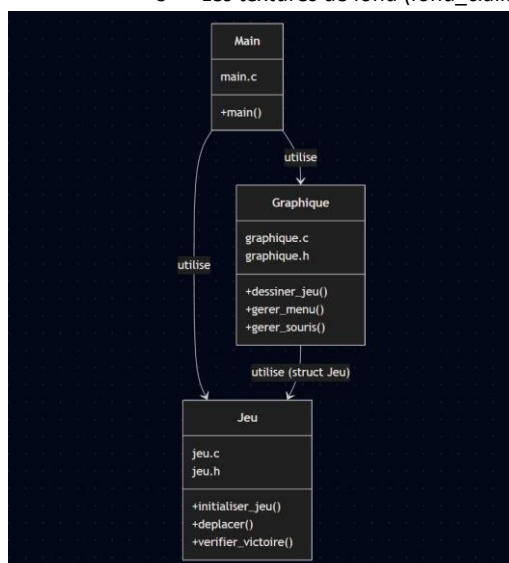
- graphique.c / graphique.h
 - Gère tout l'aspect visuel et les interactions homme-machine.
 - Contient les fonctions pour dessiner le menu de configuration (dessiner_menu) et le plateau de jeu (dessiner_jeu).
 - Gère l'affichage des images (découpage et positionnement des tuiles) et des éléments d'interface (boutons, textes, cadres de sélection).
 - Interprète les entrées souris (gerer_souris) en convertissant les coordonnées pixels en actions logiques.

3.3. Gestion de la logique

- jeu.c / jeu.h
 - Implémente la logique "métier" du Taquin, totalement indépendante de l'affichage.
 - Gère les règles du jeu : initialisation de la grille résolue, vérification de la validité des déplacements (deplacer), mélange du plateau (melanger_jeu) et détection de la victoire (verifier_victoire).
 - Définit la structure de données principale Jeu.

3.4. Répertoire des ressources

- Le dossier racine contient les fichiers graphiques nécessaires au fonctionnement du jeu :
 - Les images haute définition pour le puzzle (img1.jpg, img2.jpg...).
 - Les miniatures pour le menu (ref1.jpg, ref2.jpg...).
 - Les textures de fond (fond_clair.jpg, fond_sombre.jpg).



4. Données représentant l'état de la partie

L'ensemble des informations nécessaires au fonctionnement du jeu est regroupé dans des structures de données C. Cette approche permet de manipuler l'état complet du jeu via un simple pointeur, facilitant ainsi les appels de fonctions et la gestion de la mémoire.

4.1. Structure principale Jeu

C'est la structure centrale du projet, définie dans le fichier d'en-tête dédié à la logique du jeu. Elle contient toutes les informations persistantes d'une partie en cours :

- **Dimensions de la grille** : Deux variables entières stockent le nombre de lignes et de colonnes choisi par le joueur lors de la configuration (par exemple 4x4).
- **La Grille** : Nous avons opté pour un tableau à deux dimensions de taille fixe maximale (8x8) pour respecter les contraintes de mémoire statique. Chaque case contient un nombre entier qui représente l'identifiant du morceau d'image. La valeur zéro est utilisée symboliquement pour désigner la case vide.
- **Coordonnées du vide** : Deux variables entières mémorisent en permanence la ligne et la colonne de la case vide. Cela permet une optimisation importante : le programme n'a pas besoin de parcourir tout le tableau à chaque déplacement pour retrouver le trou.
- **Compteur de coups** : Un entier comptabilise le nombre de déplacements valides effectués depuis le début de la partie.
- **Taille visuelle des tuiles** : Une variable stocke la taille en pixels d'une case. Elle est calculée dynamiquement lors de l'initialisation en fonction de la taille de l'image source et du nombre de cases, permettant à l'interface graphique de dessiner les tuiles à la bonne échelle.

4.2. Structure de configuration Config

Une seconde structure est utilisée spécifiquement pour la phase de menu. Elle sert de tampon pour stocker les choix temporaires de l'utilisateur avant que le jeu ne soit réellement initialisé.

Elle regroupe trois informations essentielles :

- L'identifiant de l'image sélectionnée (par exemple 1, 2 ou 3).
- Le nombre de lignes souhaité.
- Le nombre de colonnes souhaité.

Cette structure permet à la fonction de gestion du menu de renvoyer l'ensemble des paramètres de configuration à la fonction principale en une seule fois.

5. Analyse des algorithmes

Cette section détaille les choix algorithmiques effectués pour assurer le bon fonctionnement et la jouabilité du Taquin.

5.1. Initialisation et Mélange (Garantie de solvabilité)

Le problème fondamental du jeu de Taquin réside dans sa solvabilité. Mathématiquement, l'ensemble des configurations possibles des tuiles se divise en deux groupes disjoints : celles qui peuvent être résolues et celles qui ne le peuvent pas. Si l'on se contentait de placer les tuiles au hasard dans la grille, le joueur aurait une chance sur deux de se retrouver face à un puzzle impossible à finir (problème de parité des permutations).

Pour contourner ce problème et garantir à 100% que chaque partie proposée soit soluble, nous avons opté pour une approche par **simulation inverse** :

1. **État initial résolu** : Le programme commence par créer une grille parfaitement ordonnée en mémoire. Les tuiles sont placées dans l'ordre croissant (de gauche à droite et de haut en bas), avec la case vide située à la toute dernière position (en bas à droite).
2. **Mélange par simulation** : Au lieu de mélanger les cases, le programme simule une partie jouée par un ordinateur très rapide. Il lance une boucle d'un grand nombre d'itérations (par exemple 500 coups).
3. **Mouvements aléatoires valides** : À chaque itération, le programme choisit une direction au hasard (Haut, Bas, Gauche ou Droite). Il vérifie si le déplacement de la case vide dans cette direction est possible (c'est-à-dire qu'elle ne sort pas des limites de la grille). Si le mouvement est valide, il est effectué instantanément.
4. **Résultat garanti** : Puisque nous partons d'un état résolu et que nous n'appliquons que des mouvements légaux (les mêmes que ceux dont dispose le joueur), nous restons mathématiquement dans l'ensemble des configurations accessibles. Il existe donc obligatoirement un chemin inverse permettant au joueur de revenir à l'état initial.

Cette méthode offre l'avantage d'être robuste et universelle, quelle que soit la taille de la grille choisie par l'utilisateur.

5.2. Gestion du déplacement

La mécanique de déplacement repose sur un échange de valeurs dans le tableau à deux dimensions. Lorsqu'une direction est demandée (par le clavier ou la souris), le programme :

- Calcule les coordonnées de la case "cible" vers laquelle le vide doit se déplacer.
- Vérifie que ces coordonnées sont bien à l'intérieur de la grille.
- Si c'est le cas, il intervertit la valeur de la case vide (0) avec la valeur de la case cible.
- Il met à jour les variables mémorisant la position du vide et incrémente le compteur de coups.

5.3. Orchestration de la partie (Boucle principale)

Contrairement à un programme classique qui s'exécute de manière linéaire, un jeu vidéo repose sur une boucle infinie, souvent appelée "Boucle de Jeu" (Game Loop). C'est le cœur du fichier principal.

Cette boucle répète inlassablement les étapes suivantes, plusieurs dizaines de fois par seconde, pour créer l'illusion d'une interaction continue :

1. **Affichage (Rendu)** : Le programme commence par dessiner l'état actuel du jeu. Pour éviter des clignotements désagréables ou une surcharge inutile du processeur, nous avons mis en place un système d'optimisation : l'écran n'est redessiné que si une modification a eu lieu (un déplacement de tuile ou un changement de menu).
2. **Gestion des événements (Inputs)** : Le programme vérifie si l'utilisateur a effectué une action. Grâce à l'utilisation de fonctions non bloquantes, le jeu est capable d'écouter simultanément :
 - **Le Clavier** : Pour détecter l'appui sur les flèches directionnelles, les touches de sélection d'image ou la touche Échap.
 - **La Souris** : Pour détecter les clics sur les tuiles ou les boutons de l'interface.
3. **Mise à jour de la logique (Update)** : Si une action valide est détectée (par exemple, un clic sur une case voisine du vide), la fonction de déplacement est appelée. La structure de données Jeu est mise à jour immédiatement : les valeurs sont échangées dans la grille et le compteur de coups est incrémenté.
4. **Vérification de la victoire** : Après chaque mouvement, l'algorithme de vérification parcourt la grille pour voir si l'ordre final (1, 2, 3... 0) est atteint. Si c'est le cas, la boucle est interrompue pour afficher l'écran de fin.
5. **Temporisation** : Une courte pause (quelques millisecondes) est insérée à la fin de chaque tour de boucle. Cela permet de libérer le processeur de l'ordinateur et d'assurer une vitesse d'exécution stable et fluide, sans consommer toutes les ressources du système.

Cette architecture permet une expérience utilisateur réactive : le joueur peut alterner à tout moment entre la souris et le clavier sans que le jeu ne se fige.

6. Conclusion personnelle

6.1. Conclusion de Rayan Ishac Belabed

Ce projet de SAE 1.1 a été une étape charnière dans mon apprentissage. Il m'a permis de consolider mes bases en langage C, notamment sur la manipulation des pointeurs qui sont indispensables pour modifier l'état du jeu à travers différentes fonctions. J'ai

particulièrement apprécié la mise en place de l'architecture modulaire. Comprendre comment découper un code en plusieurs fichiers .c et .h et les assembler avec un Makefile est une compétence que je sens essentielle pour la suite. La gestion de la bibliothèque graphique a été un défi technique (notamment l'installation sous Linux), mais réussir à créer une interface interactive avec des menus et des images est une grande satisfaction.

6.2. Conclusion de Mahdi Chaouch

De mon côté, ce projet m'a permis de mieux comprendre l'importance de la structuration des données. Définir la structure Jeu et Config avant de commencer à coder a grandement simplifié le développement des fonctions de jeu. J'ai trouvé la partie algorithmique très intéressante, en particulier la logique de mélange par simulation pour garantir la solvabilité, et la conversion des clics souris en coordonnées de grille. Travailler en binôme nous a aussi obligés à bien nous coordonner sur les noms des fonctions et des variables pour que nos codes fonctionnent ensemble, ce qui est un bon entraînement pour le travail d'équipe.